

onPanda: Efficient Annotation of On-Policy Alignment Data for LLMs and Agents via Token-Level Correction

Lei Yang¹, Mengyin Liu^{1,2}, Jia Wang¹, Hangyu Guo¹,
Liang Zhao¹, Zheng Ge¹, Kang An¹, Binxing Jiao¹,
Qi Han¹, Daxin Jiang¹, Siqi Shen², Xiangyu Zhang¹
¹StepFun ²Xiamen University

<https://on-panda.github.io/research> yanglei@stepfun.com

Abstract

We present onPanda, an interactive tool for efficiently annotating LLM alignment data and agent trajectories. onPanda adopts token-level correction as its core interaction: while reading a model response, the annotator locates the first inappropriate token and either picks a substitute from the model’s candidate tokens or types the correct text via free-form editing. The system then truncates everything after that position and continues generation from the corrected prefix, repeating this locate-correct-continue loop until a satisfactory response is obtained. This mechanism lets annotators precisely steer model outputs at low cost: a small controlled study suggests that onPanda could reduce annotation time by 52% over manual post-editing. Since the vast majority of tokens in the final response are generated by the model itself, the resulting data largely preserves the model’s sampling distribution and is well suited for constructing on-policy SFT and preference data. Furthermore, the token-level corrections recorded during annotation provide fine-grained supervision with precise positions and naturally paired positive-negative samples. onPanda also connects to external tools and harnesses, enabling interactive trajectory annotation in realistic environments. In addition, we release Panda-CVL, a dataset annotated with onPanda, together with a benchmark for token-level correction.

1 Introduction

Scaling high-quality data remains the central bottleneck for LLM alignment and agents. Existing pipelines struggle to reconcile annotation cost, on-policy fidelity, and supervision granularity: manually writing (Ouyang et al., 2022; Nakano et al., 2022) or post-editing (Yao et al., 2023; Scheurer et al., 2024; Schick et al., 2023) responses for SFT (Ranzato et al., 2016; Arora et al., 2022) is costly and yields off-policy data, while preference

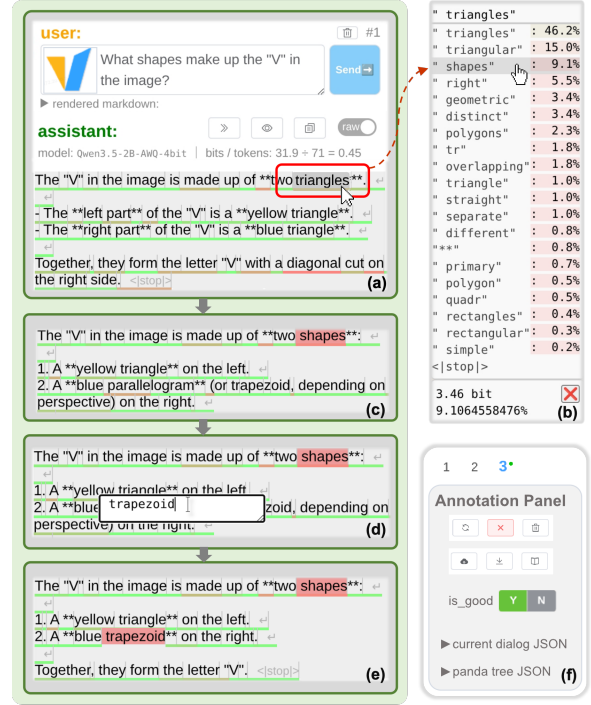


Figure 1: **The token-level correction interface of onPanda.** (a) Each token’s generation probability is color-coded below it: greener means higher, redder means lower. (b) Hovering over the first inappropriate token `triangles` pops up the top-20 candidate tokens at this position; the annotator selects the more appropriate `shapes`. (c) Replace and continue: the system replaces `triangles` with `shapes` (highlighted with a red background), truncates the subsequent content, and lets the model continue generating from the corrected prefix. (d) Free-form editing: when no candidate fits, the annotator double-clicks the erroneous `parallel` to open an input box and replaces it with the correct `trapezoid`. (e) The model continues generating after `trapezoid`, finally yielding a response that meets the SFT quality bar (`is_good=Y`). (f) The session preserves all intermediate versions: (a), (c), and (e) correspond to Dialogs [1, 2, 3]; the final Dialog 3 is the positive sample (`is_good=Y`), while Dialogs 1 and 2 are retained as negatives.

data is cheaper but offers only coarse response-level (Lightman et al., 2024; Uesato et al., 2022; Wu et al., 2023) supervision and, being confined to

candidates the model can already sample, provides little corrective signal when the model cannot sample good responses. Annotating agent trajectories is even harder: a trajectory arises from multi-step environment interaction, so any correction must actually execute tools and obtain real feedback before generation continues. Existing tools (Jurgens et al., 2026; Ou et al., 2025) can observe and score live agents and intervene mid-run, and Rep-tile (Dou et al., 2025) further lets annotators edit model-output text that its terminal environment then executes; yet the community still lacks an interactive tool that, across diverse environments, both corrects general reasoning and tool calls in real time and executes the corrected calls. We therefore develop onPanda (**on-Policy alignment data annotator**), an interactive tool for efficiently annotating alignment data and agent trajectories.

onPanda’s token-level correction interaction lets annotators precisely control and steer LLM generation, as illustrated in Figure 1. While reading a model response, the annotator only needs to locate the first inappropriate token; onPanda displays the model’s top- k candidate tokens with their probabilities at that position: the annotator simply clicks a suitable substitute if one exists, or double-clicks the token and types arbitrary replacement text when the candidates miss the correct direction. After each correction, the system truncates the subsequent content and lets the model continue from the updated prefix—hence the annotator no longer needs to re-read the entire response after a fix, but simply repeats this “locate–correct–continue” loop along the response until it meets the requirements.

This paradigm directly addresses the above bottlenecks, bringing four advantages: **(1) Efficient annotation.** Most corrections are mouse clicks, with brief typing only when the candidates fall short—a far lower cognitive and typing burden than writing from scratch or manual post-editing; the linear “correct-as-you-read” workflow also avoids repeated review of the entire response, further cutting time cost. **(2) High on-policy fidelity.** Human intervention touches only the few erroneous positions; the vast majority of tokens are generated by the rollout model itself, either sampled directly from the prompt or continued from a corrected prefix, and corrections drawn from the model’s own high-probability candidates perturb the sampling distribution even less. Unlike manual post-editing, the resulting SFT data thus stays close to the rollout model’s distribution. **(3) Fine-grained super-**

vision. Alongside the SFT data, onPanda automatically records every token-level correction, capturing the error position, the substitute token, and a naturally paired positive–negative sample. These signals are finer-grained than response-level preferences: they can be converted into preference data for reward model or DPO training, or into process reward data for PRM training, based on the step containing the corrected token and its preceding steps. **(4) Complete expressiveness.** Free-form editing is a fallback beyond candidate selection, freeing annotation from the limits of the top- k candidate set: even when the model cannot sample the correct content on its own—where preference annotation can hardly help—the annotator can still inject the correct text and steer subsequent generation. Such injections deviate from the model’s own distribution, but occur only at the few positions beyond its capability, trading a minimal distributional cost for correctness. Hence, whenever the annotator can recognize and supply the correct fix, onPanda can construct SFT samples that meet the task requirements.

For agent settings, onPanda can access tools via MCP and connect to harnesses such as Claude Code, Codex, and OpenClaw, enabling interactive trajectory annotation in realistic environments. Its intuitive interface further suits model inspection.

In addition, we release Panda-CVL (a **Chinese Vision–Language** dataset for token-level correction), annotated with onPanda, together with an accompanying benchmark, to facilitate community research on this new type of data.

Our contributions are as follows:

- We propose onPanda, an annotation tool centered on token-level correction for efficiently annotating LLM alignment data and agent trajectories that stay close to the rollout model’s distribution.
- We experimentally validate onPanda’s annotation efficiency and the on-policy fidelity of the produced data, and demonstrate its support for multimodal and agent-trajectory annotation.
- We release the multimodal Panda-CVL dataset with an accompanying benchmark, providing public resources for research on token-level correction data.

2 Design and Implementation

2.1 System Overview

onPanda is a componentized front-end library with two usage modes. As a lightweight web app, it works out of the box: deployed via static hosting without any database, it lets users drag in local annotation files and start annotating, with the browser sending requests to preset or custom Chat Completions APIs. Alternatively, its components can be embedded into an existing data platform whose backend handles task dispatching and data collection—the form in which onPanda is integrated into our in-house production annotation system. onPanda only requires the inference API to (i) continue generation from an assistant-message prefix (e.g., vLLM’s `continue_final_message`) and (ii) return each token’s top- k candidates with probabilities (logprobs). All artifacts of an annotation session—messages, annotations, operation logs, and probability caches—are serialized into a single `.panda.json` file for easy distribution, collection, and parsing.

2.2 Token-Level Correction Engine

Generation requests stream with logprobs: onPanda records each token’s sampled probability and top- k (default 20) candidates, and renders the corresponding probability color below each token (Figure 1a). Since tokenizers may split a multi-byte character or emoji across tokens, the UI groups the token stream into minimal readable units (*chunks*) along grapheme boundaries: interaction operates on chunks, while records stay token-precise. Each correction preserves three pieces of information: the correction position, the text replacement, and the samples before and after the correction.

After the annotator clicks a candidate or double-clicks to edit, onPanda truncates everything after the correction point and requests native continuation with the corrected response as the assistant prefix; tokens before the correction point are kept as-is rather than regenerated. Text introduced by free-form editing initially has no probability information; a single `prompt_logprobs` request recomputes per-token probabilities and candidates for the entire response (this feature additionally requires `prompt_logprobs` support from the API). This “refresh” also applies to arbitrary external text and cross-model settings: pasting any response into onPanda (or switching the model) reveals the current model’s confidence on every token, making

onPanda double as a model-inspection tool.

2.3 Annotation Tree and Data Protocol

An annotation session is organized as an *annotation tree* whose nodes are dialogs, each containing a complete copy of the messages and tools together with their annotations. Each correction forks a new node whose parent is the pre-correction dialog (Dialogs [1,2,3] in Figure 1 form a three-node chain: the initial rollout and two iterative corrections). All intermediate versions are thus saved automatically, without any version management by the annotator. Each node carries an operation log recording the operation type, timestamp, correction content, whether the operation is marked as on-policy, and a snapshot of the sampling configuration, giving every data point fully traceable provenance.

Each dialog also carries a quality verdict `is_good` and a project-defined annotation schema (single-choice, multiple-choice, text, etc.). The companion Python library `onpanda` parses `.panda.json` into training data: nodes with `is_good=Y` are exported as SFT samples, and positive–negative nodes under the same prompt are paired into response-level preference data. Within such pairs, the negative is typically an ancestor of the positive: the two share the prefix before the correction point and diverge exactly there, so the corresponding token-level correction data can be computed under any tokenizer as triples (negative sample, rejected token and its position, chosen token). Such data provides supervision that is precise in both position and update direction; positive and negative tokens pair one-to-one at the same position, so optimization receives naturally balanced signals. We believe these properties make token-level correction a promising cornerstone for more efficient post-training methods.

2.4 Agentic and Multimodal Annotation

Responses of reasoning models and agents are not plain text but structured messages containing reasoning, content, and `tool_calls`, which resist direct token-level correction and continuation. onPanda addresses this with the *response template* mechanism, which converts bidirectionally between structured messages and the model’s native token stream: the rendering direction produces, per the model’s response template, the full token sequence with special tokens (e.g., `</think>`, `<|tool_call_begin|>`) for display, correction, and continuation; the parsing direction restores

the generated stream into structured messages in real time for storage and tool execution. Special tokens are directly visible and correctable by annotators, so token-level correction uniformly covers reasoning chains, content, and tool-call arguments (Figure 2). Since dialogs are stored in structured form and a model’s response template is applied only during rendering and correction, the same data can be further corrected and continued by different models.

External environments and tools connect via MCP; the `harness_to_mcp` adapter wraps existing harnesses such as Claude Code, Codex, and OpenClaw into MCP servers for onPanda. Tool calls can be configured to await annotator approval before execution: a problematic call can be executed after its arguments are corrected, or rejected with optional textual guidance, and rejected trajectories are automatically kept as negative samples. Tool results are fed back into the context and generation continues, enabling interactive trajectory annotation in realistic environments. onPanda likewise supports inputting, displaying, and annotating image, audio, and video messages.

3 Evaluation

3.1 Annotation Efficiency and Data Quality

Baselines. We compare against two mainstream annotation paradigms: (1) *manual post-editing*: the answer box is pre-filled with the model’s initial rollout, which the annotator edits until it meets the SFT quality bar; (2) *preference ranking*: four rollouts are pre-generated per prompt; the annotator ranks them and records whether the best one qualifies as SFT data. The two paradigms are instantiated with the widely used POTATO and Argilla, both full-featured, freely configurable platforms that we set up as the most typical alignment-data workflows. Because each platform differs in both its interface and annotation paradigm, this comparison evaluates complete workflows, making it difficult to determine whether the observed gains arise from the interface, the paradigm, or their combination.

Setup. Three annotators labeled 21 image-description prompts, evenly split into 3 groups. A Latin-square design rotates the order of the three methods: each group is annotated exactly once per method and no annotator labels the same prompt with different methods, balancing prompt difficulty, individual proficiency, and ordering effects. None of the annotators are onPanda developers, and all

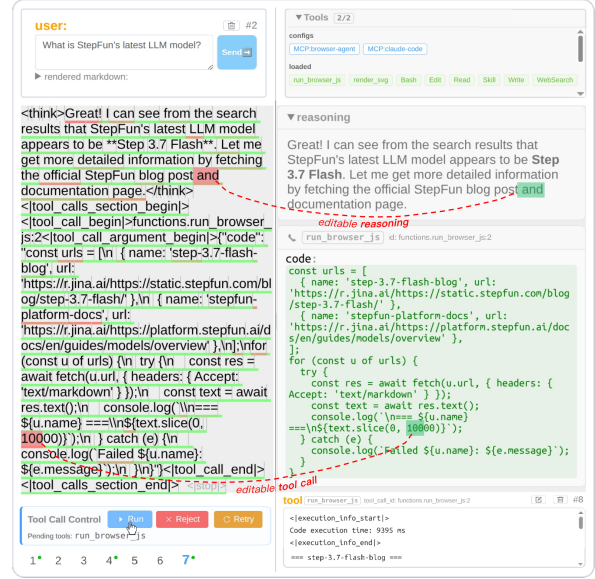


Figure 2: **Annotating an agent trajectory with onPanda.** Top left: the user prompt; top right: the connected MCP servers and tools. Left: the assistant response is rendered by the response template into the model’s native token stream, where special tokens (e.g., `</think>`) are directly visible and correctable; the annotator corrects tokens in the reasoning and in the tool-call arguments, then approves execution. Right: the modified token stream is parsed in real time into reasoning and `tool_calls`, displayed separately, with the returned tool results shown at the bottom right.

completed training and warm-up tasks on all three methods. Initial rollouts are generated by the same model, Qwen3.5-35B-A3B (instruct mode), with the officially recommended sampling parameters (temperature 0.7, top-*p* 0.8); onPanda, POTATO, and Argilla’s first candidate share the same rollout to reduce sampling randomness.

Metrics. We report five metrics. (1) *Time*: median annotation time, with the mean also reported; the median is robust to the long tail of difficult prompts, while the mean reflects total human cost. (2) *Pairwise win rate*: for each prompt, the three outputs—the final responses from onPanda and POTATO and Argilla’s top-ranked response—are compared pairwise by GPT-5.5, with each pair evaluated in both orders to reduce position bias. (3) *PPL*: response perplexity under the rollout model, used to measure on-policy fidelity. We independently sample four rollouts per prompt, use their mean PPL as the baseline, and report both each method’s PPL and its relative change (Δ PPL). (4) *SFT coverage*: the proportion of prompts that yield a qualified SFT response; under

preference ranking, all four rollouts may be unqualified. (5) *Preference pairs*: the average number of preference pairs obtained per prompt.

Results. As Table 1 shows, onPanda’s median time is 330 s per prompt, 51.5% less than POTATO’s 681 s and on par with Argilla’s 336 s; in means, onPanda (515.6 s) is 27.5% and 24.7% lower than POTATO (711.1 s) and Argilla (684.5 s). Argilla’s mean far exceeds its median, consistent with the long right tail of reviewing four lengthy responses on hard prompts. Part of the gain over POTATO stems from continuation: a hallucination or error often recurs across a response—post-editing must spot and fix every occurrence, whereas onPanda corrects only its first occurrence and the continuation stays consistent with the fix. onPanda also attains the highest pairwise win rate (66.7%), showing that the efficiency gain does not come at the cost of quality; an anonymized human comparison further supports this result: human evaluators preferred onPanda over POTATO in 54.8% of pairs.

On on-policy fidelity, onPanda and Argilla reach PPL 1.181 (Δ PPL +0.86%) and 1.161 (−0.83%), both within 1% of the baseline (1.171) and within re-sampling noise (per-prompt PPL fluctuates by about $\pm 2.8\%$ across rollouts); POTATO yields 1.596 (+36.31%). onPanda thus preserves the rollout model’s sampling characteristics much better than manual post-editing. Meanwhile, Argilla selects a qualified SFT response on only 11/21 (52%) prompts, whereas the editing-based POTATO and onPanda keep editing until the response qualifies, reaching 100% coverage by design. onPanda further derives 7.43 preference pairs per prompt, precisely aligned to correction positions, versus 6 from Argilla’s four-way ranking and 0.95 from POTATO’s pre-/post-edit pair (one initial rollout qualified without edits).

3.2 User Study and Real-World Deployment

The three annotators then filled out an adapted NASA-TLX questionnaire following (Nakano et al., 2022) for each tool, rating workload on 6 subscales from 0 to 10. We averaged the six ratings per annotator and then across annotators, with lower scores indicating lower workload. onPanda achieved the lowest score (3.1), compared with Argilla (5.4) and POTATO (6.8). One annotator remarked: “onPanda gives immediate feedback—I can see my progress on each item; the ranking workflow instead forced me to hold a very long con-

text in mind, which was mentally taxing.” Another noted: “The probability shading speeds up locating what to fix—low-probability tokens often mean the model is less confident and more error-prone, so checking them first makes annotation faster.”

Since deployment, onPanda has continuously produced three types of production data—vision, audio, and agentic—with 25,596, 105,143, and 1,257 annotation sessions respectively, automatically yielding about 388K token-level corrections (see Table 2 in appendix). Across all qualified responses, 97.0% of tokens are model-generated, 2.1% are selected from candidates, and only 0.9% are manually typed, showing sparse human intervention at production scale.

3.3 Model Inspection

onPanda supports model diagnosis through token-probability visualization. Given a selected model and arbitrary text, it displays per-token generation probabilities using color coding and entropy statistics. This enables users to determine whether a faulty rollout originates from the model or a pipeline error, inspect error probabilities and candidate distributions at critical positions, and combine probability analysis with token-level correction to steer generation along alternative paths. Together, these features provide fine-grained feedback for model behavior analysis and data quality auditing.

4 Dataset and Benchmark

Dataset. To support research on token-level correction data and its downstream applications, we construct **Panda-CVL**, a publicly releasable subset of production data annotated using onPanda. We apply two filtering criteria: (1) all images are either created in-house or obtained from openly licensed sources (see the Ethics section); and (2) the tasks do not require internal business knowledge and instead evaluate general-purpose capabilities such as visual perception, image description, and visual reasoning. Panda-CVL is a predominantly Chinese vision-language dataset comprising 7,491 annotation sessions stored in the .panda.json format, with 6,839 sessions in the training set and 652 in the test set. The auxiliary rollouts used during annotation were generated using step-1o-turbo, a 32B-parameter dense VLM.

Benchmark. A single token-level correction performed by an annotator can be decomposed into three subtasks: (1) determine whether the response

Tool	Paradigm	Time (s)↓	Win rate↑	PPL	Δ PPL	SFT Cov.↑	Pref. pairs↑	NASA-TLX↓
Argilla	rank 4 candidates	336 (684)	28.6%	1.161	- 0.83%	52% (11)	6.00	5.4
POTATO	post-editing	681 (711)	54.8%	1.596	+ 36.31%	100% (21)	0.95	6.8
onPanda	token-level correction	330 (516)	66.7%	1.181	+ 0.86%	100% (21)	7.43	3.1

Table 1: Comparison of annotation paradigms in the controlled study. Time is reported in seconds as the median annotation time over the 21 prompts, with the mean in parentheses. Win rate is the fraction of LLM-judged pairwise comparisons each method wins against the other two. PPL is scored by the rollout model; Δ PPL is the relative change from the re-sampling baseline (1.171); smaller absolute values indicate higher on-policy fidelity. SFT Cov. is the fraction of prompts yielding a valid SFT response, and Pref. pairs is the mean number of derivable preference pairs per prompt. NASA-TLX, from the user study, is on our adapted 0–10 scale (lower is better).

meets the quality standard for acceptable SFT data, in which case the annotator marks `is_good=Y` and submits it; otherwise, (2) identify the first inappropriate token in the response; and (3) correct it to an appropriate token, either by selecting from candidate tokens or by manually editing it. Following this decomposition, we evaluate models on the Panda-CVL test set by asking them to perform token-level correction in the same manner as human annotators. Given an image, a user prompt, and a candidate response, the model must determine whether the response is acceptable and, if not, locate and correct its first error. The first-error position and the human-provided replacement recorded during annotation serve as ground truth.

To support this evaluation, we design a prompt template that requires the model to produce its answer in a find-and-replace format. If the response requires no modification, the model should output `<|split|><|is_good|><|split|>`. Otherwise, it should output `<|split|>{matched_text}<|split|>{matched_index}<|split|>{replacement_text}<|split|>`. The *matched text* is a short span beginning at the first inappropriate token and is used to locate the error. The *match index* identifies which occurrence to replace when the span appears multiple times; it is 0 in most cases. The *replacement text* is a short correct span that replaces the match and begins the corrected continuation.

Evaluation Protocol. We expand the response trajectories in the 652 test conversations into 2,126 evaluation instances, comprising 652 final acceptable responses labeled as good and 1,474 intermediate responses labeled as not good. Because the evaluated models use different tokenizers, we require replacement spans to cover multiple tokens. Using the Qwen3.6 tokenizer, we compute correction triples for two pairs: (1) the ground-truth negative

response and the final acceptable response from the same conversation, and (2) the ground-truth negative response and the response obtained by applying the model-generated correction. A prediction is considered correct if the two resulting triples are identical. This procedure enables tokenizer-agnostic evaluation across models. The complete prompt template and evaluation implementation are provided in the accompanying onPanda repository.

Metrics. Table 3 reports the following metrics:

Format: The proportion of outputs that can be successfully parsed and executed, measuring the model’s instruction-following ability under the proposed template. An output that is syntactically parseable but whose matched text does not occur in the candidate response, and therefore fails to locate the target span, receives a score of 0.5.

GoodAcc: The proportion of ground-truth good instances for which the model correctly outputs `<|is_good|>`. This metric measures the model’s ability to avoid unnecessarily modifying already acceptable responses. **Loc.-NG:** The proportion of ground-truth not-good instances for which the model correctly locates the ground-truth first-error position. **Corr.-NG:** The proportion of ground-truth not-good instances for which both the located position and the replacement agree with the ground truth. This metric measures end-to-end correction performance. **F1:** The harmonic mean of GoodAcc and Corr.-NG, used as the overall metric. This aggregation follows ProcessBench (Zheng et al., 2025). Two trivial strategies—always predicting `is_good` and always attempting a correction—each yield a score of zero on one of the two component metrics and therefore obtain an F1 score of zero.

Results. We evaluate recent multimodal models on the Panda-CVL test set. Overall, token-level correction remains challenging for them. The complete results and analysis are provided in Appendix A (Table 3).

5 Conclusion

We presented onPanda, an annotation tool centered on token-level correction. Through the locate–correct–continue loop, annotators precisely steer model generation with a few clicks and edits, efficiently producing alignment data and agent trajectories that stay close to the rollout model’s distribution, while automatically retaining every correction as fine-grained supervision. Experiments and a user study show that onPanda reduces annotation time and workload while preserving data quality. We open-source onPanda and release the Panda-CVL dataset and benchmark, hoping the community will exploit token-level correction as a novel supervision signal for more efficient post-training.

Ethics Statement

Participants in our efficiency experiment and user study are full-time data annotators at our institution; the tasks fall within their regular salaried duties, and participation was voluntary and informed. Survey results are reported only in aggregate, and no personally identifiable information is disclosed in this paper. The annotation tasks involve image-description content without offensive or high-risk material. The released Panda-CVL contains only images created by us or under open licenses, manually reviewed before release to exclude personally identifiable information and inappropriate content, and is intended for research use only. As a general-purpose annotation tool, onPanda yields data whose properties depend on its users’ annotation guidelines; we encourage practitioners to follow responsible data curation practices.

References

- Argilla. Argilla: Open-source data curation and human feedback platform. <https://www.argilla.io/>.
- Kushal Arora, Layla El Asri, Hareesh Bahuleyan, and Jackie Chi Kit Cheung. 2022. [Why exposure bias matters: An imitation learning perspective of error accumulation in language generation](#). In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 700–710. Association for Computational Linguistics.
- ByteDance Seed. 2026. Seed2.1. https://seed.bytedance.com/en/seed2_1. Official model page, accessed 2026-09-17.
- Longxu Dou, Cunxiao Du, Shenggui Li, Tianduo Wang, Tianjie Zhang, Tianyu Liu, Xianwei Chen, Chenxia Tang, Yuanheng Zhao, and Min Lin. 2025. [Reptile: Terminal-agent with human-in-the-loop learning](#). GitHub repository.
- Deqing Fu, Tong Xiao, Rui Wang, Wang Zhu, Pengchuan Zhang, Guan Pang, Robin Jia, and Lawrence Chen. 2025. [TLDR: Token-level detective reward model for large vision language models](#). In *The Thirteenth International Conference on Learning Representations*.
- Spence Green, Jason Chuang, Jeffrey Heer, and Christopher D. Manning. 2014. [Predictive translation memory: A mixed-initiative system for human language translation](#). In *Proceedings of the 27th Annual ACM Symposium on User Interface Software and Technology*, UIST ’14, pages 177–187, New York, NY, USA. Association for Computing Machinery.
- David Jurgens, Michael Chen, and Lina Iyer. 2026. [Potato 2.0: A comprehensive annotation platform with AI-in-the-loop support](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 374–386. Association for Computational Linguistics.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. [Let’s verify step by step](#). In *International Conference on Learning Representations*.
- Moonshot AI. 2026. [Kimi-K2.6](#). Hugging Face model card.
- Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. 2022. [Webgpt: Browser-assisted question-answering with human feedback](#). Preprint, arXiv:2112.09332.
- OpenAI. 2026a. GPT-5.5. <https://developers.openai.com/api/docs/models/gpt-5.5>. Official OpenAI API model documentation, accessed 2026-09-17.
- OpenAI. 2026b. GPT-5.6 Sol. <https://developers.openai.com/api/docs/models/gpt-5.6-sol>. Official OpenAI API model documentation, accessed 2026-09-17.
- Tianyue Ou, Wanyao Guo, Apurva Gandhi, Graham Neubig, and Xiang Yue. 2025. [AgentDiagnose: An open toolkit for diagnosing LLM agent trajectories](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 207–215. Association for Computational Linguistics.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John

- Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. 2022. [Training language models to follow instructions with human feedback](#). In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744. Curran Associates, Inc.
- Qwen Team. 2026a. [Qwen3.5: Towards native multi-modal agents](#).
- Qwen Team. 2026b. [Qwen3.6-35B-A3B: Agentic coding power, now open to all](#).
- Marc’Aurelio Ranzato, Sumit Chopra, Michael Auli, and Wojciech Zaremba. 2016. [Sequence level training with recurrent neural networks](#). In *International Conference on Learning Representations*.
- Sebastin Santy, Sandipan Dandapat, Monojit Choudhury, and Kalika Bali. 2019. [INMT: Interactive neural machine translation prediction](#). In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP): System Demonstrations*, pages 103–108, Hong Kong, China. Association for Computational Linguistics.
- Jérémy Scheurer, Jon Ander Campos, Tomasz Korbak, Jun Shern Chan, Angelica Chen, Kyunghyun Cho, and Ethan Perez. 2024. [Training language models with language feedback at scale](#). *Preprint*, arXiv:2303.16755.
- Timo Schick, Jane Dwivedi-Yu, Zhengbao Jiang, Fabio Petroni, Patrick Lewis, Gautier Izacard, Qingfei You, Christoforos Nalmpantis, Edouard Grave, and Sebastian Riedel. 2023. [PEER: A collaborative language model](#). In *International Conference on Learning Representations*.
- StepFun. 2026. Step-3.7-Flash. <https://huggingface.co/stepfun-ai/Step-3.7-Flash>. Hugging Face model card, accessed 2026-09-17.
- Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process-and outcome-based feedback. *arXiv preprint arXiv:2211.14275*.
- Zeqiu Wu, Yushi Hu, Weijia Shi, Nouha Dziri, Alane Suhr, Prithviraj Ammanabrolu, Noah A Smith, Mari Ostendorf, and Hannaneh Hajishirzi. 2023. Fine-grained human feedback gives better rewards for language model training. *Advances in Neural Information Processing Systems*, 36:59008–59033.
- Zonghai Yao, Benjamin J. Schloss, and Sai P. Selvaraj. 2023. [Improving summarization with human edits](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2604–2620. Association for Computational Linguistics.
- Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025. [ProcessBench: Identifying process errors in mathematical reasoning](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1009–1024, Vienna, Austria. Association for Computational Linguistics.

A Additional Results and Statistics

Detailed Results. Table 3 presents the full evaluation results on the Panda-CVL test set. Token-level correction remains challenging for all evaluated models, with the best F1 score reaching only 17.09%. GPT-5.5 achieves the highest GoodAcc (53.37%) and the best overall F1, whereas Doubao-Seed-2.1-pro obtains the highest Loc.-NG (21.64%) and Corr.-NG (13.70%). GPT-5.6-sol achieves the best Format score (99.98%) but performs less strongly on GoodAcc, indicating that reliable format compliance does not necessarily translate into accurate quality assessment or correction.

Most reasoning models achieve high Format scores, while their Corr.-NG scores remain below 14. This large gap suggests that the main difficulty lies in identifying the first semantic error and producing an appropriate replacement rather than following the required output format. The substantial variation in GoodAcc also shows that models differ considerably in their tendency to accept or modify candidate responses. Finally, the Instruct baseline performs poorly across all metrics, suggesting that reasoning-oriented inference is particularly important for token-level correction.

B Related Work

Annotation tools for LLMs and agents. General-purpose platforms such as POTATO (Jurgens et al., 2026) and Argilla (Argilla) can be configured for post-editing or preference annotation, and POTATO 2.0 further supports online and offline agent-trajectory annotation. Closest to our work is Reptile (Dou et al., 2025): targeting terminal-based SWE tasks, it lets annotators locally edit model-output text to trigger continuation, which its terminal environment parses and executes before resuming the rollout, thereby yielding SFT data with higher on-policy fidelity than full manual post-editing. However, Reptile’s corrections rely almost entirely on text manually typed by annotators rather than selected from the model’s own candidate tokens, inevitably limiting the data’s on-policy fidelity; its current implementation also encodes reasoning and actions as plain text and shell commands, and cannot support general tool-call formats. By contrast, onPanda combines selecting from probability-scored candidate tokens with continuation over structured reasoning and tool calls, covering plain-text

Metric	Vision	Audio	Agentic
<i>Scale</i>			
Annotation sessions	25,596	105,143	1,257
Annotators	32	42	24
Total human-hours	3,557.4	18,871.3	523.5
Annotation time P25 (min) [†]	0.64	0.95	17.82
Annotation time P50 (min) [†]	1.65	1.93	31.31
Annotation time P75 (min) [†]	4.48	3.81	50.01
<i>Data yield</i>			
SFT samples (is_good=Y) [†]	1.001	1.318	6.018
Token-level corrections [†]	2.256	3.034	8.566
Candidate clicks [†]	1.684	2.655	5.410
Double-click edits [†]	0.572	0.380	3.156
Regenerations [†]	0.003	0.544	0.303
<i>Per is_good response (avg.)</i>			
Reasoning corrections	—	—	0.34
Content corrections	2.24	2.05	0.05
Tool-call corrections	—	—	0.66
Zero-correction rate (%)	49.1	23.9	53.5
Model-generated tokens	207.0	34.7	557.0
Candidate-selected tokens	1.6	1.6	0.9
Annotator-typed tokens	1.0	0.7	1.2
<i>Task profile</i>			
Image inputs [†]	1.026	—	52.010
Audio inputs [†]	—	1.000	—
Tool calls [†]	0	0	5.230
User turns [†]	1.000	16.163	1.387

Table 2: Production annotation statistics for the Vision, Audio, and Agentic onPanda deployments. [†] denotes statistics reported per annotation session. Some projects require annotating multiple SFT instances from a single session to increase data throughput. Furthermore, a single agent trajectory contains multi-turn tool calls that can serve as SFT data. Due to the nascent state of agentic annotation and unoptimized rollout models, stricter control is required, leading to increased manual typing.

LLM, multimodal, and agent data, and explicitly records each correction’s position, replacement text, and pre-/post-correction samples. We do not compare against Reptile directly because it targets terminal-based SWE tasks with different environments and action representations; we compare workflows rather than benchmark the systems head-to-head on a shared task.

Fine-grained supervision signals. For fine-grained supervision, PRM800K (Lightman et al., 2024) collects step-level labels through human annotation, while TLDR (Fu et al., 2025) constructs token-level labels from synthetic perturbations. In contrast, onPanda’s token-level correction signals are position-precise and naturally paired as positives and negatives, arising directly from the annotator’s process of constructing qualified responses, without extra human labeling or synthetic perturbation.

Model	Format	GoodAcc	Loc.-NG	Corr.-NG	F1
<i>Reasoning</i>					
Doubao-Seed-2.1-pro(ByteDance Seed, 2026)	99.01	9.66	21.64	13.70	11.33
GPT-5.5 (xhigh)(OpenAI, 2026a)	93.96	53.37	15.26	10.18	17.09
GPT-5.6-sol (xhigh)(OpenAI, 2026b)	99.98	16.56	19.95	13.09	14.63
Kimi-K2.6(Moonshot AI, 2026)	92.41	26.40	18.47	10.59	15.11
Qwen3.5-397B-A17B(Qwen Team, 2026a)	93.77	20.40	17.37	10.99	14.28
Qwen3.6-35B-A3B(Qwen Team, 2026b)	68.58	26.23	9.57	5.70	9.36
Step-3.7-Flash(StepFun, 2026)	78.83	40.09	5.81	3.32	6.13
<i>Instruct</i>					
Qwen3.5-397B-A17B(Qwen Team, 2026a)	38.83	4.45	1.63	0.81	1.38

Table 3: Scores on the Panda-CVL test set (%). The best result in each column is shown in bold.

Correct-and-continue interaction. The pattern of “correcting a prefix and regenerating the suffix” predates the LLM era: Predictive Translation Memory (Green et al., 2014) and INMT (Santy et al., 2019) already applied it to interactive machine translation. onPanda brings this mechanism into general-purpose alignment-data annotation and extends it to multimodal and structured agent scenarios, exporting branches of the annotation tree as three data types: SFT, preference, and token-level correction.

Limitations

The core interaction of onPanda relies on two inference-API capabilities: continuing generation from an assistant-message prefix, and returning top- k candidate logprobs (probability refresh further requires `prompt_logprobs`). Some proprietary APIs do not expose both, so onPanda cannot annotate on-policy data for the models behind them. In practice this constraint is mild: official APIs from many vendors—including the proprietary Doubao, as well as DeepSeek, Kimi, and StepFun—already provide both prefix continuation and logprobs; for open-weight models, mainstream inference frameworks such as vLLM and SGLang natively support all required features, llama.cpp covers the core interaction, and vLLM-based hosted services such as W&B Inference¹ work with onPanda out of the box. Moreover, reasoning and tool-call continuation for a new model requires adapting its response template; with our reference implementations and development guide, a coding agent can usually complete the adaptation automatically, though it remains a one-time integration cost.

Second, the efficiency and on-policy advantages of token-level correction rest on a sparse-error premise: the model should produce responses whose local logic is largely sound, so that human intervention is needed only at a few positions. When

the model falls far short of the target task and corrections become dense, both advantages diminish; annotators can then fall back to free-form editing, which onPanda also supports. Moreover, the on-policy property is relative to the rollout model at annotation time: its benefit fades when the data is used to train a different model, or as the trained model’s weights are continually updated.

Finally, our empirical evidence has its own boundaries: the controlled experiment is modest in scale (3 annotators $\times$ 21 image-description prompts, a single rollout model), and the user-study participants all come from our in-house annotation team; quality evaluation primarily relies on LLM-as-a-judge, supplemented by a small human comparison between onPanda and POTATO; automated judgments may still reflect judge-specific preferences despite position-swapped comparisons. We do not conduct a controlled agent study; the agentic evidence is limited to system capability and deployment statistics. More importantly, this paper validates annotation efficiency, output quality, and the distributional properties of the produced data, whereas the downstream training benefit of token-level correction signals remains unverified by training experiments.

C Future Work

Post-training on token-level correction data.

Token-level correction data offers supervision that is precise in both position and direction, with naturally paired positive–negative signals. To verify its downstream training benefit, we are developing post-training methods built on such data: the *token-level correcting model*, a token-level counterpart of reward models, and *token-level correction optimization*, a token-level counterpart of DPO. The benchmark released with this paper evaluates exactly this correction capability, providing a ready testbed for the correcting model.

Annotation–training flywheel. A correcting model can in turn assist annotation by pre-locating suspicious tokens and proposing fixes for annotators to review. Moreover, onPanda can be integrated with a training pipeline into a closed loop: each newly annotated batch triggers an incremental training step and updates the served rollout model, which then assists subsequent annotation. As the model improves, corrections grow sparser and annotation keeps getting cheaper, while the data stays on-policy with respect to the latest

¹<https://wandb.ai/inference>

weights—mitigating the on-policy decay noted in Limitations—so that annotation and training accelerate each other.

Online learning from textual feedback. on-Panda’s upcoming support for annotating earlier turns will yield data of a new form: guided by feedback that arrives in later turns (e.g., tool errors or user complaints), annotators revise an earlier rollout—future feedback supervising past generation. This mirrors the supervision structure of online learning from textual feedback, where deployed agents continually receive compiler errors, tool failures, and users’ textual corrections, all pointing to concrete mistakes in earlier outputs. Historical-turn annotations could therefore become a natural data source for this direction: models (e.g., the correcting model) trained to translate such feedback into token-level corrections of earlier rollouts would let LLMs learn continually from real interactions without scalar rewards. In this vision, token-level correction serves as a unified intermediate representation that turns arbitrary textual feedback—from annotators, environments, or end users—into trainable supervision.